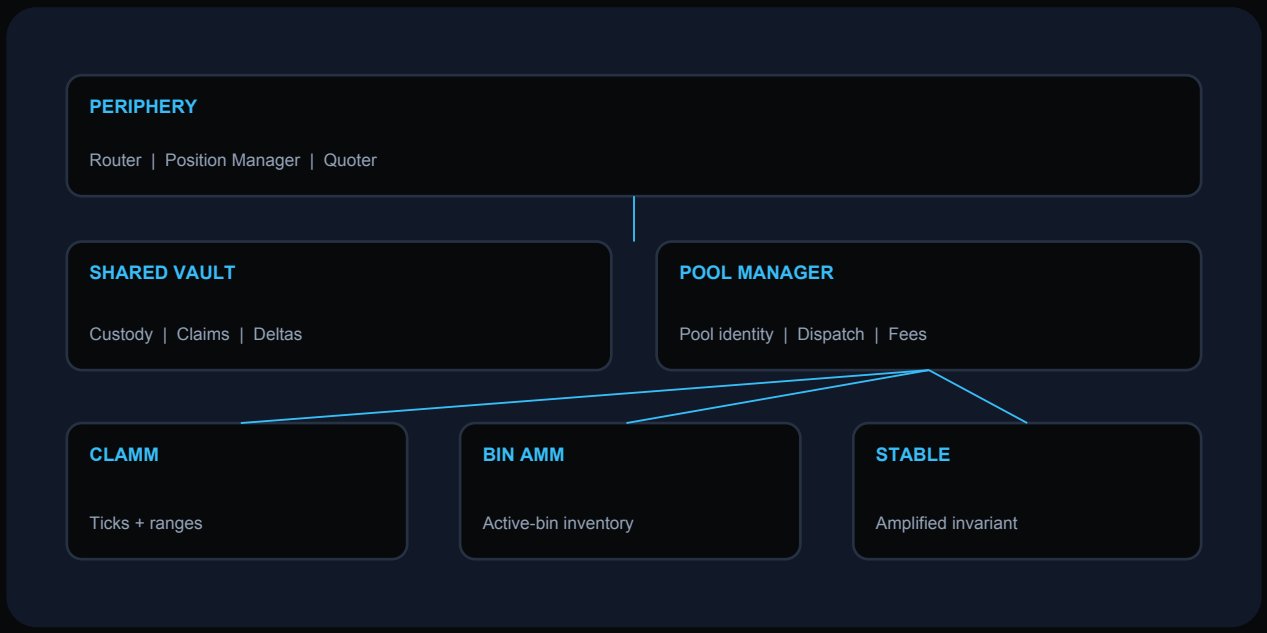
SUPADAO DEX / TECHNICAL PUBLICATION

SupaDAO DEX White Paper

A unified singleton exchange architecture for multiple liquidity regimes



VERSION 1.0.0 · 15 OCTOBER 2026 · UNDER TECHNICAL REVIEW

Repository-derived documentation. No independent audit or public deployment was confirmed.

Document control

This publication is reproducible from Markdown sources in the SupaDAO DEX repository. It describes inspected source and is not legal, tax or investment advice.

Contents

01 Executive summary
02 1. Why multiple liquidity models?
03 2. AMMs in plain English
04 3. Liquidity mechanisms
05 4. Architecture
06 5. Swap lifecycle
07 6. Accounting and fees
08 7. Usage scenarios
09 8. Security architecture and residual risk
10 9. Testing and deployment status
11 10. Limitations
12 11. Roadmap by validation stage
13 12. Conclusion
14 References
15 Glossary
16 Revision record
17 Risk notice

White paper version: 1.0.0 Publication date: 2026-10-15 Document status: Under Technical Review Scope: Repository implementation reviewed at publication time

SupaDAO DEX is independent from the SupaCore institutional settlement project. This document describes the DEX only. No independent security audit or public-network deployment was confirmed during this review.

Executive summary

SupaDAO DEX is an experimental decentralized exchange implementation that coordinates multiple automated-market-maker designs through one shared manager and custody/accounting vault. A pool selects one of three specialized engines: concentrated liquidity, discretized liquidity bins, or a two-asset amplified stable invariant. Pool identity also includes currencies, fee, spacing and plugin address, so different pools for the same token pair remain economically distinct.

"Singleton" here describes shared coordination and settlement, not a single monolithic contract and not automatic liquidity aggregation. SupaPoolManager validates and dispatches pool operations; SupaVault custodies tokens, represents internal balances as ERC-6909 claims and requires transient currency deltas to net to zero before an unlock session ends. Specialized engine contracts retain curve-specific state and mathematics.

The architecture aims to reduce duplicated exchange infrastructure while preserving liquidity models suited to different assets. It also provides plugins for dynamic fee experiments and anti-JIT residency logic. The LVR mechanism should be read as an experimental adaptive-fee design, not insurance: its internal oracle uses pool ticks rather than an external fair-price oracle, and current post-swap wiring does not update the plugin's stored tick. Its economic effect is therefore not established.

The repository contains Solidity implementations, Foundry tests, deployment scripts, a router, quoter, position NFTs, circuit breakers and timelock administration. This is meaningful engineering evidence, but no independent audit, verified public testnet deployment, mainnet deployment or production operation was established.

1. Why multiple liquidity models?

An AMM replaces a continuously maintained order book with deterministic state transitions. Traders pay assets into a pool and receive assets out according to a curve; liquidity providers supply the inventory and generally earn a portion of swap fees. The curve decides how liquidity is distributed across possible prices.

No single distribution dominates every market. Concentrated liquidity can place capital inside selected ranges but demands range management. Discrete bins represent inventory at price levels and can offer fixed-price execution within a bin, but require rules for allocation and traversal. A stable invariant can reduce slippage near parity for economically similar assets, but becomes sensitive to depegs, decimal/rate assumptions and parameter calibration.

Building a separate exchange contract for every model duplicates custody, routing, fee logic and security boundaries. Deploying them on top of one settlement layer aims to preserve specialized mathematics while sharing custody and the swap execution envelope.

2. AMMs in plain English

A constant-product pool prices trades using $x \cdot y = k$. An input increases one reserve and reduces the other so their product remains constant, subject to fees. Price is the marginal exchange rate of the two balances.

Concentrated liquidity bounds that relationship to a price range $[\sqrt{P_l}, \sqrt{P_u}]$. Providers supply assets only for prices inside that interval. Capital is utilized more intensely while market price remains inside the range,

but yields zero fees and shifts entirely into the depreciating asset if price leaves it.

Discrete bins treat price levels as discrete buckets. Assets placed at a specific price can be swapped at that price until the bucket is empty. StableSwap combines constant-sum and constant-product behavior through an amplification factor A. When reserves are balanced, trades behave closer to $x + y = \text{const}$; as reserves diverge, the curve behaves closer to $x \cdot y = k$, protecting the pool from complete depletion.

Each mechanism trades off capital efficiency, operational complexity, depeg exposure and adverse selection.

3. Liquidity mechanisms

Concentrated liquidity (CLAMM)

CLAMMEngine tracks square-root prices in Q64.96 format, maps prices to integer ticks, uses a tick bitmap to find the next initialized tick, and accrues fee growth inside each range. Active liquidity scales token quantities during swaps. Crossing a tick updates liquidity and fee accumulators.

Discretized liquidity bins (BinAMM)

BinAMMEngine maps price levels to bins and tracks reserveX and reserveY for each active bin. A swap consumes available opposite-asset reserves in the active bin at that bin's discrete price. The reviewed implementation executes against the active bin; automated traversal to neighboring bins was not present.

StableSwap (StableAMM)

StableAMMEngine implements an amplified invariant for two assets using iterative Newton-Raphson approximations. It calculates the invariant D during liquidity operations and target balance y during swaps. The engine assumes tokens trade at 1:1 and does not include rate or decimal normalization.

Dynamic LVR and anti-JIT plugins

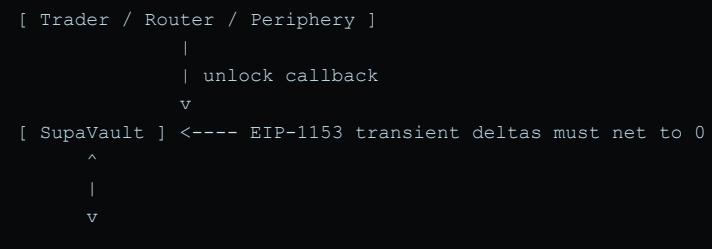
Loss-versus-rebalancing represents the expected loss of a passive liquidity provider relative to continuous rebalancing against an external market price. It is distinct from ordinary price slippage, smart-contract exploits and impermanent loss. SupaDAO DEX includes an experimental LVRShieldPlugin intended to increase swap fees during volatility using a ring buffer of historical pool observations.

Because the oracle derives its signal from internal pool ticks rather than an external oracle, and because post-swap wiring in the reviewed source does not update the stored tick, this plugin should be evaluated as an experimental adaptive-fee prototype, not a proven guarantee against arbitrage.

An AntiJITVestingPlugin tracks liquidity deposit timestamps and burns a decaying penalty if liquidity is withdrawn before a minimum residency window expires.

4. Architecture

SupaDAO DEX uses a singleton coordination and settlement architecture with delegated curve computation:



```
[ SupaPoolManager ] ---- plugin lifecycle & circuit-breaker checks
      |
      +---> [ CLAMMEngine ]      (Ticks, ranges, fee growth)
      |
      +---> [ BinAMMEngine ]     (Active-bin inventory)
      |
      +---> [ StableAMMEngine ]  (Amplified invariant)
```

PoolKey(currency0, currency1, fee, tickSpacing, plugin, curveType) is hashed into a unique PoolId. The manager enforces initialization, circuit-breaker checks, plugin permissions and curve dispatch. SupaVault custodies ERC-20 tokens, tracks internal balances as ERC-6909 claims, and requires all transient currency deltas to net to zero before an unlock session finishes.

5. Swap lifecycle

1. A trader sends an exact-input or exact-output request to SupaRouter, including a deadline, path and slippage bound.

2. The router opens a vault unlock session. The vault records the router as locker in transient storage.

3. The router decodes the path and calls SupaPoolManager.swap for each hop.

4. The manager derives PoolId, checks pause state and initialization, then asks the plugin for any fee override.

5. The selected curve engine computes amount deltas, changes curve state and accrues LP/protocol fees.

6. The manager runs the after-swap plugin callback, posts token deltas to the vault and synchronizes price/liquidity summaries.

7. The router pays inputs and takes outputs using tokens, native currency or claims according to the call path.

8. The vault verifies that no transient currency delta remains; otherwise the entire transaction reverts.

Different engines share the envelope but not the invariant. That distinction is central: unified accounting does not make CLAMM ranges, bins and the amplified stable curve mathematically interchangeable.

6. Accounting and fees

Pool fees use hundredths of a basis point (1e-6), so 3000 represents 0.30%. Swap code separates a configured protocol share and adds the LP portion to curve-specific fee-growth accumulators. PositionManager "pokes" a position with zero liquidity change to synchronize fees, then represents amounts owed as ERC-6909 claims before collection.

Protocol fees accrue by currency in the manager. Only the configured controller can collect them. SupaFeeCollector can distribute collected values to DAO treasury, LP staking and insurance reserve allocations once enabled by governance.

7. Usage scenarios

• Trader: A user selects a route and supplies a minimum output. The relevant engine prices the trade; plugin fees may change the effective fee. The transaction reverts if the deadline or output bound fails. Risks include price movement, MEV, insufficient liquidity, hook failure and asset behavior.

• Concentrated LP: A provider chooses ticks and deposits required token amounts. Capital can be efficient within range, but inactivity and one-sided exposure occur outside it. Fees depend on actual trade flow and active

liquidity.

• Bin LP: A provider currently interacts with active-bin accounting. Available reserves cap the output; adjacent-bin traversal is not present in the reviewed swap path.

• Stable-pool LP: Two assumed-equivalent assets are deposited into an amplified pool. Near-balance execution may be flatter, but decimal mismatch, rate drift and depeg risk are material because no normalization/rate-oracle logic is included.

8. Security architecture and residual risk

The vault's callback lock and transient zero-delta check protect the settlement boundary. Authorized managers alone can post third-party deltas. Routers enforce deadlines and slippage limits. Solidity 0.8.26 checked arithmetic is used except where libraries justify unchecked regions. Circuit breakers can pause vault, curve or pool scopes; guardians pause and the owner unpauses. A custom timelock can delay administration. Only the vault is UUPS upgradeable in current source.

Residual risks include implementation errors in custom math, rounding at boundaries, malicious/non-standard tokens, hook reentrancy or griefing, governance or upgrade-key compromise, parameter abuse, oracle manipulation, liquidity manipulation, MEV, depegs and denial of service. Tests and formal-style properties improve confidence but do not replace an independent audit, public bug bounty, deployment rehearsal or operational controls.

9. Testing and deployment status

Foundry is configured for Solidity 0.8.26, Cancun, IR compilation, optimizer runs of 1,000,000, fuzz runs of 1,024 and invariant campaigns of 256 runs by depth 64. The repository contains unit, fuzz, invariant, integration, benchmark, formal-style and deployment-script sources.

Current tests verify isolated pool mechanics, multi-curve math boundaries, fee splits, claims routing, NFT generation and circuit breakers on local Anvil networks. No independent audit, verified public testnet deployment, mainnet deployment or live TVL has been established.

10. Limitations

• The LVR plugin's tick/oracle wiring limits the intended adaptive signal.

• The bin engine does not traverse adjacent bins in the reviewed swap function.

• The stable engine lacks decimal/rate normalization and supports two assets.

• A universal pair-wide route optimizer was not found.

• TVL valuation is not a canonical on-chain protocol primitive.

• Plugin/oracle write permissions and configuration require additional threat review.

• Only the vault implements UUPS upgradeability; administrative centralization persists until a verified handoff.

• No independent audit or public deployment was confirmed.

11. Roadmap by validation stage

• Completed in repository: Core contracts, three engine types, periphery, claims, governance controls, local deployment scripts and broad Foundry suites.

• Required before public testnet: Resolve documented engine/plugin limitations, reproduce full tests and gas snapshots, establish canonical addresses, publish deployment configuration and conduct targeted adversarial review.

• Required before mainnet readiness assessment: Independent audit, remediation and retest; public testnet observation; token-compatibility policy; operational key/guardian procedures; monitoring, incident response and a public bug bounty.

12. Conclusion

SupaDAO DEX's proposition is architectural: share custody, settlement and lifecycle infrastructure while selecting curve logic per pool. That can simplify integrations and make different liquidity regimes available through a common envelope, but it concentrates correctness and governance requirements in shared components. The next credible milestone is not a broader performance claim; it is resolving known limitations and validating the system independently on a public testnet.

References

1. Milionis, Moallemi, Roughgarden, Zhang, "Automated Market Making and Loss-Versus-Rebalancing," 2022/2024, https://arxiv.org/abs/2208.06046

2. Egorov, "StableSwap - efficient mechanism for Stablecoin liquidity," 2019, https://docs.curve.finance/assets/pdf/whitepaper_stableswap.pdf

3. Adams et al., "Uniswap v4 Core Whitepaper and Concepts," 2024, https://docs.uniswap.org/contracts/v4/concepts

4. Solidity documentation, https://docs.soliditylang.org/en/v0.8.26/

5. Foundry invariant testing documentation, https://getfoundry.sh/forge/invariant-testing

6. EIP-1153, Transient storage opcodes, https://eips.ethereum.org/EIPS/eip-1153

7. EIP-6909, Minimal multi-token interface, https://eips.ethereum.org/EIPS/eip-6909

Glossary

• AMM: On-chain mechanism that quotes trades from deterministic state.

• CLAMM: AMM with liquidity assigned to price ranges.

• Invariant: Mathematical relationship the swap transition preserves or solves.

• LVR: Adverse-selection loss relative to continuous rebalancing.

• MEV: Value extractable through transaction inclusion or ordering.

• Singleton: Shared coordination/accounting instance for many pools.

• Slippage: Difference between expected and executed price as size/state changes.

• TVL: Value assigned to deposited assets, requiring an explicit valuation methodology.

- Liquidity bin: Inventory assigned to a discrete price level.
- StableSwap: Amplified invariant family intended for correlated-value assets.

Revision record

Version	Date	Status	Change
1.0.0	2026-10-15	Under Technical Review	Initial repository-derived white paper publication.

Risk notice

Decentralized exchange interactions carry financial, technical, market, execution and smart-contract risks. This document is provided for technical information only and does not constitute financial, legal, investment, or tax advice. No warranty is made regarding the protocol's security or suitability for any purpose.